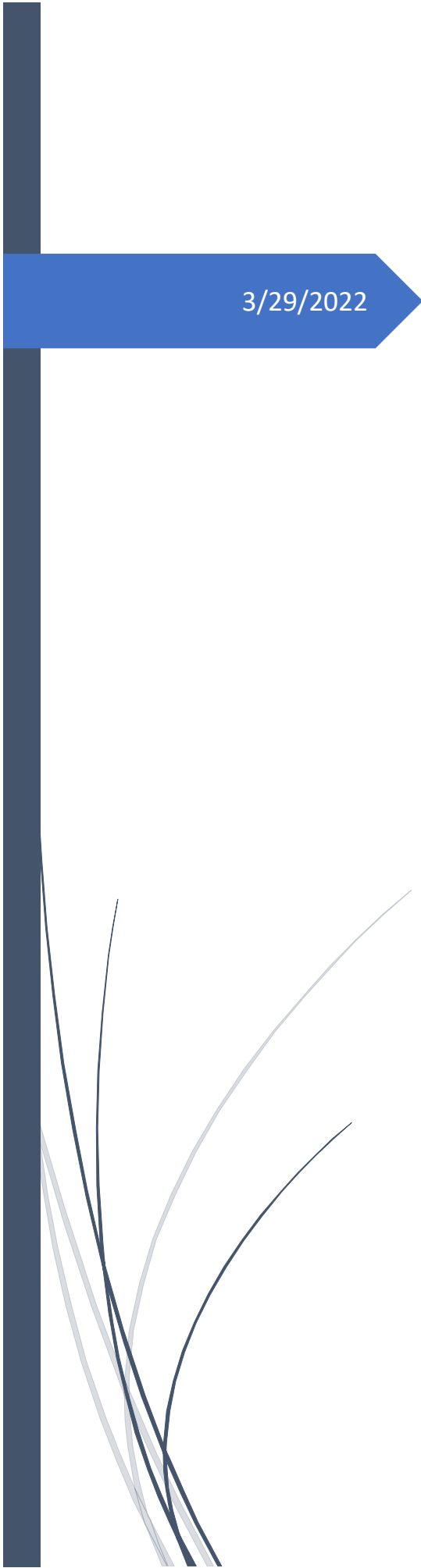




3/29/2022

F28HS: Hardware-Software Interface

Assessed Coursework 2 — Systems Programming

Farhan Shahriar & Nura Tamton
GROUP 25

Course code and name:	F28HS HARDWARE SOFTWARE INTERFACE
Type of assessment:	Group
Coursework Title:	Assessed Coursework 2- System Programming
Student Name:	Nura Tamton
Student ID Number:	H00378175

Declaration of authorship. By signing this form:

- **I declare** that the work I have submitted for individual assessment OR the work I have contributed to a group assessment, is entirely my own. I have NOT taken the ideas, writings or inventions of another person and used these as if they were my own. My submission or my contribution to a group submission is expressed in my own words. Any uses made within this work of the ideas, writings or inventions of others, or of any existing sources of information (books, journals, websites, etc.) are properly acknowledged and listed in the references and/or acknowledgements section.
- I confirm that I have read, understood and followed the University's Regulations on plagiarism as published on the [University's website](#), and that I am aware of the penalties that I will face should I not adhere to the University Regulations.
- I confirm that I have read, understood and avoided the different types of plagiarism explained in the University guidance on [Academic Integrity and Plagiarism](#)

Student Signature (type your name): *Nura Tamton*

Date: *29/03/2022*

Course code and name:	F28HS – Hardware-Software Interface
Type of assessment:	Group

Coursework Title:	Assessed Coursework 2 — Systems Programming
Student Name:	Farhan Shahriar
Student ID Number:	H00339163

Declaration of authorship. By signing this form:

- **I declare** that the work I have submitted for individual assessment OR the work I have contributed to a group assessment, is entirely my own. I have NOT taken the ideas, writings or inventions of another person and used these as if they were my own. My submission or my contribution to a group submission is expressed in my own words. Any uses made within this work of the ideas, writings or inventions of others, or of any existing sources of information (books, journals, websites, etc.) are properly acknowledged and listed in the references and/or acknowledgements section.
- I confirm that I have read, understood and followed the University's Regulations on plagiarism as published on the [University's website](#), and that I am aware of the penalties that I will face should I not adhere to the University Regulations.
- I confirm that I have read, understood and avoided the different types of plagiarism explained in the University guidance on [Academic Integrity and Plagiarism](#)

Student Signature (*type your name*): *Farhan Shahriar*

Date: *29/03/2022*

Contents

1. Problem Specification	4
2. Hardware Specification and Wiring	4
3. Code Structure	4
master-mind.c	4
lcdBinary.c	6
test.sh	6
testm.c	6
4. Execution of the program in debug mode	7
5. Summary	7
6. Video	8

1. Problem Specification

In this coursework, we were required to implement a simple instance of the Mastermind board-game, using C and ARM assembler as implementation language. The application had to run on a Raspberry Pi 3, with the given devices, one button and two LEDs. Mastermind is a two-player game between a codekeeper and a codebreaker. The Raspberry Pi should act as codekeeper, so it needs to generate a (secret) random sequence and answer with the number of exact and approximate matches in the sequence above. The user acts as the codebreaker and must enter the guess sequence. The user inputs the guess using the button and after each round of guess, the LEDs must blink accordingly to communicate with user. The LEDs will blink as specified to inform the user whether the inputs were accepted. Moreover, after each round of guess the LEDs will blink the same number of times as the exact and approximate match. The game is over once the user successfully guesses the secret sequence. The program would run-on debug mode to display the secret sequence.

2. Hardware Specification and Wiring

This Master-Mind game is implemented using a Raspberry Pi. A single button was used as an input device and two LEDs (green and red) were used as the output. All three hardware were connected to the pi via a breadboard. The first LED (green) was linked to the pi by connecting the positive side of the LED to GPIO 13, while the second LED (red) was connected to GPIO 5. The button was connected to GPIO 19. Three resistors were used, one for each of the hardware devices to control the current, along with three male/male wires. Five male/female wires were used to set up the button and the LEDs. Three of them connecting to each of the GPIO pins, whereas the last two wires were used to connect to ground and power supply pin.

3. Code Structure

`master-mind.c`

Majority of the codes implemented to run the game is contained in this file. This file consists of the C codes needed to generate a secret sequence and to read the guess sequence inputted by the user, which were then compared to check whether the user entered the correct guess.

`void initseq()`

This function was used to generate three random numbers from 1 to 3.

```
void showSeq(int *seq)
```

The purpose of this function is to display the secret code generated when in debug mode.

```
int *countMatches(int *seq1, int *seq2)
```

This function checks the secret sequence and the guess sequence and compares the two. The exact number of matches and the approximate number of matches are returned.

```
void showMatches(int *code, int *seq1, int *seq2, int lcd_format)
```

This function prints the exact and approximate values on the terminal.

```
void readSeq(int *seq, int val)
```

The integer values get parsed and stored in seq.

```
int readNum(int max)
```

This function will read the standard input and store them into the array.

```
uint64_t timeInMicroseconds()
```

This function was implemented with `gettimeofday()` and converts the time taken into microseconds.

```
void initTimer(uint64_t timeout)
```

This is function used as a timer with the parameter timeout, that determines how long the timer will run.

```
void blinkN(uint32_t *gpio, int led, int c)
```

This function is used to make the led blink as many times as required. The function `writeLED()` consists of the value parameter which determines whether the LED should be on or off.

```
int getInput()
```

The purpose of this function is to count the number of times the button was pressed. Using the `waitForButton()` function to wait for the button to be pressed and once the button has been pressed, the timer starts and the function goes into a while loop which will stop once the stopT (stop time) is reached. The `readButton()` will then read the button input and store it, which is then returned to the function.

```
int main(int argc, char *argv[])
```

This is where we call the functions that we implemented to run the game. The configuration of the LEDs and button was done on the main function along with the memory mapping. The

codes to control the GPIO pins used for the hardware devices are implemented here. The main loop that we implemented starts by counting the number of times the button was pressed in each round and each guess for that round is stored into an array. Then it checks with the secret code that was generated to find the exact and approximate matches. Furthermore, the LEDs then start blinking according to the number of times specified in the Mastermind game. Lastly, the SUCCESS message is printed if the user was able to accurately guess the secret code.

lcdBinary.c

All the functions in this file were first coded in C, and then translated to assembly. This file consists of the functions that directly interact with the hardware (Button, LEDs)

```
void pinMode(uint32_t *gpio, int pin, int mode /*, int fSel, int shift */)

```

In this function, the mode of the GPIO pins was set to Input or Output. In this case the GPIO pins connected to the LEDs are set as the output and the GPIO connecting to the button was set as the input.

```
void writeLED(uint32_t *gpio, int led, int value)

```

This function sets the value which is either LOW or HIGH to the LED GPIO pins.

```
int readButton(uint32_t *gpio, int button)

```

This function is used to read the LOW or HIGH value from the GPIO pin connecting to the button. Therefore, it is used to check if the button has been pressed.

```
void waitForButton(uint32_t *gpio, int button)

```

This function waits for a button input and is implemented using readButton().

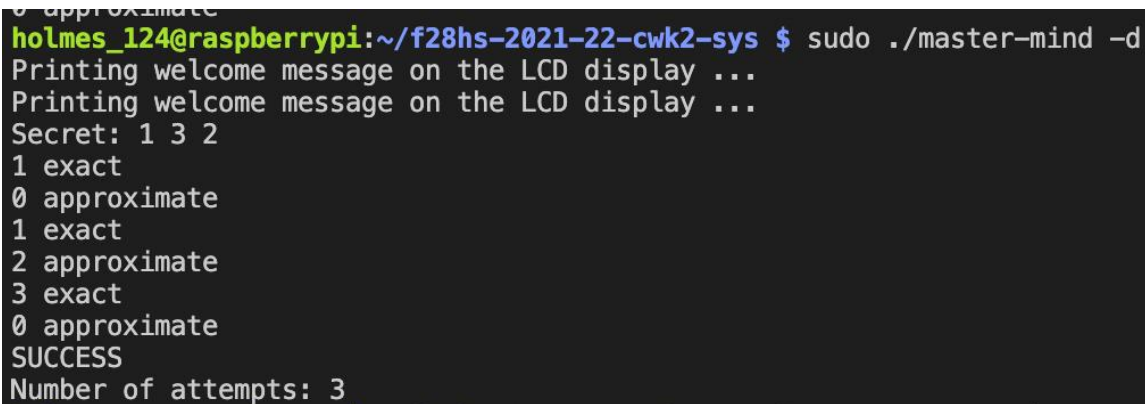
test.sh

This file is used to test the matching function in C.

testm.c

This function tests the matching functions used in C and in assembly and compares the two.

4. Execution of the program in debug mode



```
0 approximate
holmes_124@raspberrypi:~/f28hs-2021-22-cwk2-sys $ sudo ./master-mind -d
Printing welcome message on the LCD display ...
Printing welcome message on the LCD display ...
Secret: 1 3 2
1 exact
0 approximate
1 exact
2 approximate
3 exact
0 approximate
SUCCESS
Number of attempts: 3
```

Figure 1: Execution of the program in debug mode in the terminal

As shown in the image above, the secret code generated was 1, 3, 2. The first guess sequence (1, 1, 1) was inputted using the button and resulted in 1 exact match and 0 approximate matches. The second round of guess (2, 3, 1) resulted in 1 exact and 2 approximate matches. Finally, the last round of guess inputted resulted in all three inputs being the exact match. This then prints out the message SUCCESS along with displaying the number of attempts it took to guess the secret code correctly.

5. Summary

Overall, the coursework achieved almost everything that was required. C and Assembly language was used to implement everything requested from the game. Through various testing and implementation of functions, we gained greater understanding of programming in C. In addition, we gained an insight into ARM Assembly programming.

One aspect that could be improved is the input of the button. For example, if the button is pressed too quickly multiple times, then it is only registered once. Furthermore, if the button is pressed for too long, multiple inputs are taken. Additionally, we were unable to compute the approximate matches in assembly as we did not fully comprehend the concepts of assembly. Due to this, 8 out of the 10 tests in testm.c passed.

6. Video

The following link contains a video demo of running the Mastermind game.

[IMG_6711.MOV](#)